```
}
}
```

7.4 File serialization

Another variant of the file I/O world is something called serialization. To serialize an object means to take an object in memory and write that object to a file. Then, at a later time, the object can be de-serialized and then we have the object—with its state intact—back in memory.

```
String myString = new String( "Some important text" );
File myFile = new File( "C:/myFile.ser" );
FileOutputStream out = new FileOutputStream( myFile );
ObjectOutputStream s = new ObjectOutputStream( out );
s.writeObject( myString );
s.flush();
```

To start, we create an object of type String "myString". Next, notice we are using a special class called an ObjectOutputStream. This class is designed to serialize objects. By custom, files of serialized objects end in ".ser". Finally, we see that we are writing an object.

The process to read from an existing Serialized file is very similar.

```
File myFile = new File( "C:/myFile.ser" );
FileInputStream in = new FileInputStream( myFile);
ObjectInputStream s = new ObjectInputStream(in);
String myString = (String)s.readObject();
```

Notice, when you read out the object from serialized file, you need to cast the object back into the type you know it is.

7.5 Reading user input from the console

Although it should be easy, you have to consider the user inputting values from the console as reading from a stream. Before we look at the program, let's understand the issues involved:

- Now do we tell it to read?
- How do we tell it to stop reading?

You tell it to read a line by hitting the "return" key on your keyboard. To tell it when to stop reading, we need to send in a "sentinel" value. That means, no matter what, stop when you read this "sentinel" value.

```
String temp = null;
boolean keepReading = true;
InputStream is = System.in;
```